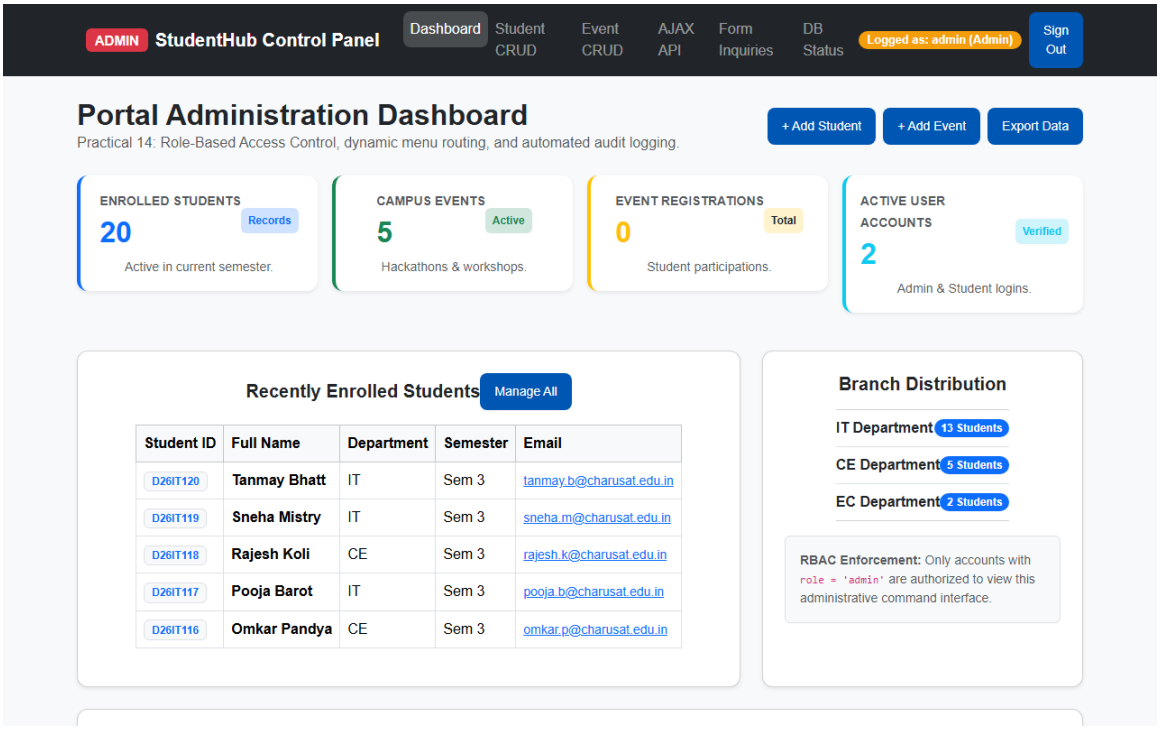
PRACTICAL – 14

Problem Definition: Develop an admin dashboard that displays user statistics, student count, event count and latest registrations. Restrict access to admin users only and show dynamic menu options based on role.

Output:

Admin Dashboard with Role-Based Access Control & Audit Log Trail:



Question & Answer:

1. Are dashboard values loaded dynamically from the database?

Ans. Yes. All statistics displayed on the dashboard—including total students, active campus events, event registrations, and user accounts—are calculated dynamically from the database using SQL COUNT() and GROUP BY queries.

2. Is access restricted to admin users?

Ans. Yes. Access is guarded by require_role('admin'). If non-admin users or unauthenticated visitors attempt to open the dashboard, access is denied and a 403 Forbidden page is served.

3. Are role-based menus working?

Ans. Yes. The navigation bar dynamically adapts based on the user's role ($_SESSION['role']), displaying administrative tools (CRUD modules, Audit Logs, Data Export) only to authorized administrators.

4. Are important admin actions recorded in audit_logs?

Ans. Yes. Key administrative events—such as logins, student additions, event modifications, and dashboard visits—are automatically logged into the audit_logs table with user ID, action name, IP address, and timestamps.

Conclusion / Learning Outcome:

In this practical, an administrative command dashboard was built for StudentHub. Role-Based Access Control (RBAC) was strictly enforced, dynamic role-sensitive navigation menus were deployed, aggregate metrics were compiled from the database, and an automated audit logging system was implemented to provide complete security traceability.

Github link :- https://github.com/07vish/sh.git